

Contents

Activity 1- Socratic Prompting	2
Recon summary	3
Activity 2 – COT	4
Analysis	4
Activity 3 – Scenario prompt	8
Response	10

Activity 1- Socratic Prompting

I want to build a scalable, data-driven Selenium automation framework.

Do not provide the implementation immediately. Instead, act as a Solution Architect and gather my requirements using the Socratic questioning technique.

Rules:

- * Ask exactly one question at a time.
- * Each question should provide multiple-choice options (A, B, C, D...) along with a brief explanation of each option.
- * Wait for my response before asking the next question.
- * Ask only the questions necessary to fully understand my requirements.
- * Do not make assumptions unless I explicitly ask for recommendations.

Once all the requirements are gathered:

1. Summarize my chosen requirements.
2. Explain the architecture and design decisions.
3. Provide the recommended project structure.
4. Implement the complete framework step by step with well-commented Java code.
5. Explain why each design choice was made and discuss possible alternatives.
6. Follow industry best practices for Selenium, TestNG, Maven, Page Object Model, and data-driven testing.

Recon summary

Project purpose: A full-stack tool ("User Story → Tests") that converts a user story (title, description, acceptance criteria, additional info) into AI-generated test case specs via the Groq LLM API. It generates structured test cases (as data/JSON rendered in a UI table) — it does not execute or automate tests itself.

Frontend: frontend/ — React 18 + Vite 5 SPA, TypeScript, no router/state library. Single 717-line App.tsx component with 5 useState hooks; form on the left, results table on the right.

Backend: backend/ — Express 4 + TypeScript + Zod. One real route POST /api/generate-tests: validates request via Zod, builds a prompt, calls Groq's chat-completions endpoint, validates the LLM's JSON response via Zod again, returns test cases + token usage. No database, no auth.

Root: npm workspaces monorepo (frontend, backend), orchestrated via concurrently.

Java/Selenium/TestNG/Maven: None found. No .java files, no pom.xml, no references to "selenium" or "testng" anywhere in the repo (source or docs) — confirmed via targeted grep and glob searches (excluding node_modules noise, which contained none either).

"Test cases" meaning: The repo produces test case *specifications* (structured JSON: title, steps, expected results, etc., displayed in a UI table) generated by an LLM from a user story — it's a test-case generator/authoring tool, not a Selenium/TestNG execution framework. There's no existing consumer that turns these into runnable Selenium code; that would be new functionality if desired.

Activity 2 – COT

Scenario:

Customer tries to transfer ₹50,000.

Transaction failed.

Error shown: "Transaction could not be completed."

Expected discoveries:

1. Daily limit exceeded
2. Insufficient balance
3. Bank server unavailable
4. OTP expired
5. Network timeout
6. Beneficiary inactive
7. Fraud rule triggered
8. Core banking unavailable
9. UPI switch unavailable

Analysis

Here's a structured root-cause diagnostic chain for this failure — organized so a support/QE team can triage systematically rather than guess.

Step 1: Classify the error by layer

The generic message "Transaction could not be completed" hides the real cause. First split the 9 possibilities into layers:

Layer	Possible Causes
Customer/Account-side	Daily limit exceeded, Insufficient balance

Layer	Possible Causes
Auth/Session	OTP expired
Network/Client	Network timeout
Beneficiary/Compliance	Beneficiary inactive, Fraud rule triggered
Infrastructure (Bank)	Bank server unavailable, Core banking unavailable
Infrastructure (Rail)	UPI switch unavailable

Step 2: Diagnostic chain-of-thought (in triage order)

Q1 — Did the customer receive/enter OTP?

- No OTP received / expired before entry → **OTP expired**
- OTP entered but still failed → move to Q2

Q2 — Was the failure instant or after a delay/spinner?

- Instant (before hitting network) → check local validation: **Daily limit exceeded** or **Insufficient balance** (these usually surface with a specific reason code, not a generic error — so also flag this as a *UX/error-mapping defect* if the generic message is shown here)
- Delayed / spinner then timeout → move to Q3

Q3 — Is this failure isolated to one customer or widespread?

- Widespread across many customers/transactions at the same time → move to Q4 (infra-level)
- Isolated to this one customer/beneficiary → move to Q5 (account-level)

Q4 — (Widespread) Check infra logs by hop

- App/client logs show request never left device → **Network timeout**
- Bank's API gateway/server logs show no response within SLA → **Bank server unavailable**
- Core banking system logs show queue backlog or DB unresponsive → **Core banking unavailable**
- NPCI/UPI switch status page or the switch's response code (e.g., timeout codes) → **UPI switch unavailable**

Q5 — (Isolated) Check account/beneficiary/risk logs

- Beneficiary status flag = inactive/deregistered → **Beneficiary inactive**
- Fraud/AML engine log shows a rule hit (velocity, amount threshold, new beneficiary + high value, etc.) → **Fraud rule triggered**

Step 3: Practical checklist to run per incident

1. Pull the transaction reference/UTR number and trace it across: App logs → Bank gateway logs → Core banking logs → UPI switch logs (NPCI).
2. Check the actual response/error code returned by the failing hop (each of the 9 causes usually maps to a distinct code — the generic message is likely a fallback/catch-all, which is itself worth flagging as a defect).
3. Correlate timestamp with any known incident/outage window (helps rule in/out #3, #8, #9 quickly).
4. Check if it's reproducible for the same customer with a smaller amount (rules in/out #1 and #2 fast).
5. Check OTP delivery/verification logs separately from the transaction logs (SMS gateway delay is a common false trigger for "OTP expired").

Step 4: Test design implication

If you're building test cases from this (given the QE angle), each of the 9 causes needs:

- A **negative test case** simulating that specific failure (e.g., mock core banking timeout, mock expired OTP)
- An **assertion on the error message shown to the user** — right now all 9 collapse into one generic message, which is a UX/traceability gap worth raising as a defect on its own, since it makes production triage much harder (support can't tell which of 9 causes without digging through logs).

Want me to turn this into a formal RCA/test-case matrix (as a table or spreadsheet) mapping each cause → detection point → test case → expected error code?

RCA & Test Case Matrix — ₹50,000 Fund Transfer Failure ("Transaction could not be completed")							
Purpose: map each candidate root cause to its detection point, a negative test case, expected error code/message, and severity — to close the generic-error traceability gap.							
#	Layer	Root Cause	Detection Point / Log Source	Test Case (Pre-condition + Action)	Expected System Behavior	Expected Error Code	Expected User-Facing Message
1	Customer/Account	Daily limit exceeded	App/API validation log; limit-check service response code	Pre-condition: customer's cumulative daily txn total is within ₹500 of their daily limit. Action: attempt a transfer of ₹50,000 that breaches the limit.	Transaction blocked pre-debit; no amount deducted; limit-check service returns a distinct decline reason.	ERR-LIMIT-001	"Daily transfer limit exceeded. You can transfer up to ₹X more today."
2	Customer/Account	Insufficient balance	Core banking balance-check response; ledger snapshot at txn time	Pre-condition: account balance < ₹50,000 (net of hold/lien amounts). Action: attempt the ₹50,000 transfer.	Transaction blocked pre-debit; no partial debit; balance-check fails fast (<2s).	ERR-BAL-002	"Insufficient balance to complete this transaction."

3	Auth/Session	OTP expired	OTP service log; timestamp delta between OTP generation and verification	Pre-condition: valid txn initiated, OTP generated. Action: wait beyond OTP validity window (e.g., >3 min), then submit the OTP.	OTP verification service returns expiry-specific code; transaction not debited; customer prompted to resend OTP.	ERR-OTP-003	"OTP has expired. Please request a new OTP."
4	Network/Client	Network timeout	Client app network logs; request never reaches server / no response within client timeout threshold	Pre-condition: simulate poor connectivity (throttled/dropped network) via proxy or airplane-mode toggle mid-transaction. Action: submit transfer.	Client detects no response within timeout (e.g., 30s), transaction marked pending/unknown, reconciliation job later confirms final state (no duplicate debit).	ERR-NET-004	"Network issue. Please check your connection and try again."
5	Beneficiary/Compliance	Beneficiary inactive	Beneficiary master status flag; beneficiary service response	Pre-condition: beneficiary account is deregistered/frozen/inactive at receiving bank. Action: attempt transfer to this beneficiary.	Beneficiary validation fails before debit leg is triggered; sender not debited.	ERR-BENE-005	"Beneficiary account is inactive. Please verify beneficiary details."
6	Beneficiary/Compliance	Fraud rule triggered	Fraud/AML rules engine decision log (rule ID, risk score)	Pre-condition: configure a scenario matching a known fraud rule (e.g., high-value first-time beneficiary, velocity breach). Action: attempt the ₹50,000 transfer.	Rules engine returns a hold/decline decision; transaction blocked or routed to manual review queue; audit trail captures rule ID.	ERR-FRAUD-006	"This transaction could not be processed. Please contact customer support."
7	Infrastructure (Bank)	Bank server unavailable	API gateway health check / 5xx response from bank's transaction server	Pre-condition: simulate bank server downtime in a test/staging environment (kill service or mock 503). Action: attempt transfer during the outage window.	Gateway returns 5xx/service-unavailable; client shows retry-safe generic error; no debit occurs; alert fires on monitoring dashboard.	ERR-SVR-007	"Service temporarily unavailable. Please try again later."
8	Infrastructure (Bank)	Core banking unavailable	Core banking system (CBS) connectivity/queue logs; DB/CBS heartbeat	Pre-condition: simulate CBS downtime or DB connection pool exhaustion in staging. Action: attempt transfer while CBS is unreachable.	Request queued/rejected at orchestration layer before reaching CBS; no debit; transaction marked failed with distinct CBS-down code; reconciliation confirms no ledger entry.	ERR-CBS-008	"Unable to process your request right now. Please try again shortly."
9	Infrastructure (Rail)	UPI switch unavailable	NPCI/UPI switch response code (e.g., timeout/switch-down codes); PSP logs	Pre-condition: simulate UPI switch non-responsiveness (mock NPCI timeout) in staging. Action: attempt a UPI-rail transfer of ₹50,000.	PSP receives switch-timeout code; transaction status set to 'deemed' pending reconciliation per NPCI TAT rules; auto-reversal triggered if applicable.	ERR-UPI-009	"UPI service is temporarily unavailable. Your money is safe; please check status after some time."

Activity 3 – Scenario prompt

Scenario 1 — Search Functionality

✗ Weak: "Test the search feature."

What's missing: No product type (e-commerce, hospital),
no filter/sort behavior, no empty/invalid input scenarios, no performance angle.

Scenario 2 — File Upload

✗ Weak: "Generate test cases for file upload."

What's missing: No file types, no size limits,
no success/failure criteria, no security angle (malicious file upload).

Scenario 3— Shopping Cart

✗ Weak: "Test the cart functionality."

What's missing: No add/remove/update actions,
no price calculation validation, no coupon/discount scenarios, no out-of-stock handling.

Scenario 4 — Role-Based Access Control

✗ Weak: "Write test cases for user roles."

What's missing: No role types (Admin, Manager, Viewer),
no permission matrix, no unauthorized access scenarios, no app domain.

Scenario 5 — API Testing

✗ Weak: "Write API test cases for user registration."

What's missing: No HTTP method, no request/response structure,
no status codes to validate, no tool context (Postman/RestAssured).

Scenario 6 — Mobile App (OTP Login)

✗ Weak: "Test OTP login on mobile."

What's missing: No platform (Android/iOS), no OTP expiry time,
no resend behavior, no network failure scenario, no Appium/device context.

Scenario 7 — Dashboard / Report Generation

✗ Weak: "Test the reports section."

What's missing: No report type (PDF/Excel/CSV), no date range filters,
no data accuracy validation, no permission-based visibility.

Scenario: [chosen scenario]

I - Instruction :

C - Context :

E - Example :

P - Persona :

O - Output Format :

T - Tone :

Response

Since you've laid out the template but not picked a scenario, I'll build out **Scenario 1 — Search Functionality** as a worked example — happy to do the same for any of the other six.

Scenario 1 — Search Functionality (E-commerce)

I – Instruction: Generate a comprehensive set of test cases for the search functionality of an e-commerce website, covering functional, boundary, negative, and performance scenarios.

C – Context: The application is an e-commerce platform (similar to Amazon/Flipkart) with a global search bar in the header. Search should support product name, brand, and SKU lookups, with autosuggest, filters (price, category, rating, brand), sorting (relevance, price low-high, price high-low, newest, rating), pagination, and a "no results found" state. The search index refreshes every 15 minutes, and the site must handle up to 500 concurrent search requests without degradation.

E – Example: Provide one sample test case in this format so I follow the same structure for the rest:

TC ID	Title	Precondition	Steps	Test Data	Expected Result	Priority
TC_SEARCH_01	Search with valid keyword returns matching products	User is on homepage	1. Click search bar 2. Enter "running shoes" 3. Press Enter	Keyword: "running shoes"	Results page shows only products matching "running shoes", sorted by relevance	High

P – Persona: Act as a Senior QA Engineer writing test cases for a sprint regression suite, reviewed by a QA Lead before being added to the test management tool (e.g., Zephyr/TestRail).

O – Output Format: A table with columns: TC ID, Title, Preconditions, Steps, Test Data, Expected Result, Priority. Group cases under headers: Functional, Boundary/Edge, Negative/Invalid Input, Filter & Sort, Empty Results, Performance/Load, Security (e.g., SQL injection via search box).

T – Tone: Precise, professional, and unambiguous — written the way a QA Lead would expect in a test case document, no filler language.

Want me to actually generate the full test case table from this prompt, or fill in the ICE-POT for another scenario (File Upload, RBAC, API, OTP Login, or Reports) first?